

Dawnwood Interactive

Optimized kernel formalization, literal-source supplement,
and measured RTX 5070 Ti performance

Native binding v0.3, optimized edition | 16 September 2026
Project author: Tom Klootwijk

Measured outcome	Result
BC5 circulation, larger field	3.56x faster / 8.61 billion pair updates/s
RG8 circulation, larger field	3.13x faster / 6.78 billion pair updates/s
BC5 capacity exercised	7.516 billion stored pairs / 10.500 GiB payload
Preserved behavior	Complete reference/optimized snapshots agree on all 16 small cases; graph checks and continuation checks pass.
Theory verdict	Digital self-reference and component algebra are supported. Physical double-slit behavior is not validated.

What this document adds

The supplied 24-page *double-slit-theory.pdf* is a conversation containing design requirements, interpretations and predictions. This edition turns the executable parts into an explicit finite numerical law, traces them to source pages, and reports measurements from the actual laptop. Numerical choices are identified as implementation definitions, not equations quoted from the source.

The optimized recurrence preserves the existing log-polar/Klein representation, one-bit jitter, field-directed mutation, exact B/A words and fourth-slot Y-up event. It also preserves their numerical limitations. In particular, logarithmic radius wrapping can amplify tiny cancellation residues; RG8 does not remove that effect.

Reading guide: pages 2-4 define the source relationship and state law; page 5 formalizes the optimization; pages 6-8 explain speed, VRAM and cache metrics; pages 9-10 assess theory fidelity and correctness; pages 11-12 give reproduction and references.

Literal source to executable law

Page numbers below refer to the supplied source PDF, not this report. Literal requirements and assistant predictions are distinguished; neither is experimental evidence.

Source pages / requirement	Executable supplement and boundary
3-4 / two pinions, Hadamard hinges, log-polar LUT	Two LP8 complex streams; $H(a,b)=((a+b)/\sqrt{2}),(a-b)/\sqrt{2})$. A finite chart and cell sizes are defined by this edition. No spatial slit aperture is specified.
4-6 / one-bit parity, jitter, [0,2,0,1]	Packed bit operations, deterministic hash-derived jitter, quantized amplitude-two seed, zero B and phase-inverse identity. Jitter is reproducible, not quantum randomness.
7 / two Y-up shifts in fourth RK slot	Fourth derivative receives $\epsilon=2/65536$ in its y coordinate. This is an intentional forced four-stage map, not textbook RK4 for the unforced ODE.
8-9 / shapes, double-dot, phase difference	Local SDF supports and real two-vector contraction. T union, triangle, cone meridian, ambient 4D sphere support and apex are explicit numerical bindings.
9-12 / R,G with B history and inverse T in A	R/G are token streams. Per 16 pairs, B is a 32-bit history and A packs two 16-bit inverse phase words. A does not invert the complete recurrence.
12-14 / self-referential SDF Klein surface	Klein quotient with local SDF supports; rectangular CUDA arrays are storage. No global signed distance to a Klein bottle or physical square lattice is asserted.
14-16 / moving, changing operators at Psi	31 mutable records; field results alter body words, references, support, coefficients, positions and routing. The CUDA machine-code kernel itself stays fixed.
16-22 / texture cache and VRAM saturation	Native BC5 textures, surface commits, a rolling whole-field sweep and capacity accounting. Cache retention is a hardware preference; allocation is not bandwidth saturation.
22-23 / universality and application claims	Programmable finite state feedback exists. There is no universality proof, task reward, learned objective, database implementation or physical quantum computer.

The detailed derivation and page map are maintained in **docs/THEORY_SUPPLEMENT.md** and **results/optimization/THEORY_AUDIT.md**. The original source PDF is unchanged.

State, storage and geometry

Let $F_e = (P_e, O_e, B_e, A_e, e, c_e, \text{seed})$. P is the stored two-stream field, O is the mutable operator catalogue, e is the Psi interval and c is the rolling block cursor. There are N physical 4x4 blocks; $W = \min(\text{requested window}, N)$.

```
i(t) = (c_e + t) mod N,      0 <= t < W
S_t = E(P_e, O_e, B_e, A_e, i(t), e, seed)
P, B, A <- commit all S_t to unique owned blocks
O_(e+1) = M(O_e, S, e, seed)
c_(e+1) = (c_e + W) mod N;  e <- e + 1
```

Evaluation reads the last committed field. A separate kernel commits the staged window; mutation reads the old operator bank and staged results; an ordered final kernel advances the clock. This prevents same-kernel surface-write/texture-read coherence assumptions. Four kernels are replayed as a CUDA Graph. [R2]

Representation	Bytes / role
Operator record	64 bytes; 31 records per bank, two alternating banks. Eight 32-bit instruction words per body.
BC5 block	16 bytes for 16 R/G token pairs, plus 8 bytes exact B/A: 1.5 bytes per pair.
RG8 block	32 bytes for 16 R/G token pairs, plus 8 bytes B/A: 2.5 bytes per pair.
Staging	64 bytes per active block; 4 MiB at $W=65,536$. No second full-size field.
Hot data	6,080 bytes shared per CTA; optimized immutable pair table adds 1 MiB in device memory.

LP8 decode and the Klein quotient

```
h = q >> 4; p = q & 15
u = (h - 0.5)/15; v = (p + 0.5)/16 [h > 0]
z(q) = 2^(-4 + 7.5*u) * exp(i*2*pi*v)
h == 0 denotes exact zero amplitude
(u+1, v) ~ (u, -v); (u, v+1) ~ (u, v)
```

Encoding a nonzero z uses $u = (0.5 \log_2(|z|^2) + 4)/7.5$ and $v = \arg(z)/(2\pi)$, folds the quotient, then selects a containing cell. Exact zero returns token 0 before offsets. Radial wrapping identifies magnitudes differing by $2^{7.5}$, approximately 181.02. This is a computational quotient, not a physical amplitude scale.

```
tau=2*pi; r(v)=2+0.5*cos(tau*v)
K(u,v)=(r(v)*cos(tau*u), r(v)*sin(tau*u),
        0.5*sin(tau*v)*cos(pi*u),
        0.5*sin(tau*v)*sin(pi*u))
K(u+1, -v)=K(u,v)
```

The complete interval

Selection, body execution and forced chart motion

The payload chooses two page-chain directions by default. A depth-four implicit binary decision tree chooses a leaf from operators 15-30 using SDF sign, jitter, history and routing parity. The leaf executes its two referenced bodies, then its own body, on eight 16-bit registers. Arithmetic overflow is modular. Operator reference 31 aliases record 0.

```
f(u,v) = (drive_u + k*sin(2*pi*v),
          drive_v + k*cos(2*pi*u))
a=f(q); b=f(q+dt*a/2); c=f(q+dt*b/2)
d=f(q+dt*c+(0,2/65536))
q_next=q+dt*(a+2*b+2*c+d)/6
```

The drive and coupling k come from the executed register words and operator coefficient. The phase offset also includes the wrapped second difference of the sampled neighbour phases. The fixed fourth-stage event is retained from source page 7. As dt approaches zero, the effective derivative tends to $(5 f(q)+f(q+(0,\epsilon)))/6$, not exactly $f(q)$.

Double pinion and exact auxiliary words

```
z_plus  = (z_R + z_G)/sqrt(2)
z_minus = (z_R - z_G)/sqrt(2)
twist   = dt*(Re(z_R)*Re(z_G) + Im(z_R)*Im(z_G))
R_next  = C(z_plus, du, qa+twist) XOR jitter
G_next  = C(z_minus, -du, qb-twist) XOR jitter
```

C is the LP8 quotient encoder with chart offsets. qa and qb are quantized phase rotations including inverse- T feedback. A stores their modular additive inverses: $p+(-p)=0 \bmod 65,536$. B rolls its previous history together with jitter, return parity, output parity and a mixed epoch word. A excludes Hadamard mixing, twist, codec loss, radial drift and the rest of the update.

Self-reference has an executable meaning

Each operator selects a staged sample using its old location, history, index and epoch. A hash of field feedback and prior operator state selects a body-word bit; jitter controls its flip. Related bits change links, support and coefficients. Positions move by the defined chart law. The next interval executes those altered bodies. This is field-dependent program-data mutation, with no fitness objective or proof that changes improve a task.

Work per interval is proportional to $W(H+D+3L+16C_{\text{codec}})+31$, with $D=4$ and $L=8$. A complete field sweep is at least linear in N . Changing W changes the mutation schedule and therefore the model, so every fair speed comparison fixes W .

Optimization without a new law

Memoize finite token-pair mathematics

```
P[r+256*g] = (U(z_plus), V(z_plus),
              U(z_minus),V(z_minus))
U(z)=(0.5*log2(|z|^2)+4)/7.5
V(z)=atan2(Im(z),Re(z))/(2*pi)
P: 65,536 entries * 16 bytes = 1,048,576 bytes
```

The table is built once on the GPU from the same uploaded 256-entry LP8 LUT. Zero branches retain an explicit sentinel. At runtime the original evolving offsets and dot product are applied in the same operation order. Sixteen V4 reads replace up to 64 log2/atan2 evaluations per 16-pair block. No evolving operator or phase is frozen. Fused multiply-add stays disabled in both builds.

Change	Why it is safe / measured cost
Exact BC5 selector search	Monotone midpoint comparisons preserve the original lowest-index tie rule. Absolute reconstruction error is accumulated while encoding.
Bounded VM arithmetic	Only operations whose ranges fit are narrowed; final 16-bit modular results are unchanged.
Address calculation	One conditional subtraction replaces window modulo under $c < N$ and $W \leq N$. Power-of-two page sides use shifts/masks; other sides retain division.
Commit error reduction	Warp shuffles and four shared partial sums replace the 128-entry reduction. Partial windows keep all CTA lanes participating.
Balanced cache preference	Prevents excessive L1 preference from starving shared-memory residency on this device. Estimated resident CTAs/SM: 1 to 8. This remains a driver preference.
Resource and evidence reporting	Reports parameters, kernel variant, cache preference, staging/table bytes, registers and estimated occupancy. Reference binary remains available.

The compiler reports 100 to 61 registers per evolve thread. Shared hot data remains 6,080 bytes. Local stack rises from 192 bytes in the original build to 208 bytes in the updated builds; reported spill stores/loads are zero. Stack allocation and compiler spills are different measurements.

The default comparison includes both kernel changes and the cache policy change. With both binaries explicitly set to balanced, the measured gains are **1.32x BC5** and **1.30x RG8**. These isolate the implementation changes more closely. [R3]

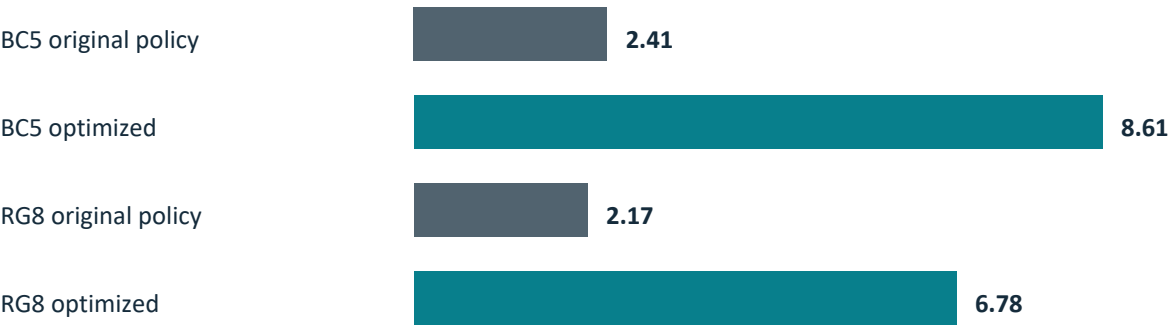
Performance, in plain language

Think of a token pair as one two-color item. Updates per second tells you how many items the machine revises; it does not count independent simulated universes, floating-point operations, frames, or newly stored items.

Workload / codec	Reference ms	Optimized ms	Speedup
2 x 1024 ² / BC5	423.23	112.37	3.77x
2 x 1024 ² / RG8	458.80	151.83	3.02x
32 x 2048 ² / BC5	889.45	249.56	3.56x
32 x 2048 ² / RG8	989.82	316.74	3.13x
Both balanced / BC5	329.99	249.74	1.32x
Both balanced / RG8	411.37	316.74	1.30x

Each row is the median of five timed runs with alternating build order and excluded process warmups. Small-field runs use 1,024 intervals; the other rows use 2,048. All use W=65,536, batch=32, seed=756, dt=1/64, two hops, jitter and mutation enabled. BC5 and RG8 are different recurrences because BC5 is lossy; compare builds within a codec.

Billion pair updates per second: larger field



What the stopwatch includes

Circulation wall time and CUDA-event time include launch gaps and batch synchronization. They exclude context creation, allocation, initialization, report output and snapshots. Full process speedups on the larger field are 2.63x BC5 and 2.37x RG8; those smaller numbers reflect startup costs.

BC5 optimized larger-field variation: median 249.56 ms; range 249.40-250.82 ms; median absolute deviation 0.165 ms.

RG8 optimized larger-field variation: median 316.74 ms; range 316.39-317.34 ms; median absolute deviation 0.099 ms.

The laptop ran under Windows WDDM, without externally fixed clocks or power limits. These are measured workloads on this machine, not a universal speed guarantee or a statistical significance claim.

VRAM capacity and sweep rate

VRAM is the large shelf; cache is the small nearby workbench. Filling the shelf is a storage result. A full sweep means every stored block has been revisited. A token may be updated several times in one run.

Measured capacity	BC5	RG8
4096 ² pages	448	269
Payload GiB	10.500	10.508
Stored pairs, billions	7.516	4.513
Free after packing, MiB	266.0	258.0
Payload / total device	87.94%	88.01%
Payload / initially free	97.59%	97.66%
Margin-run payload GiB	7.875	7.852
Repeated margin-run speedup	3.89x	3.34x
Optimized billion updates/s	8.26	6.54

The maximum-allocation run used fill=0.98 and reserve=256 MiB. A later repetition at that maximum failed during allocation. Three successful alternating trials therefore used 75% of the observed page counts: 336 BC5 / 201 RG8 pages, each for 8,192 intervals. The last three rows describe this margin workload. Full snapshots were not exported at these sizes.

```
payload bytes/pair = 1.5 (BC5) or 2.5 (RG8)
updates/s = 16 * visited_blocks / circulation_seconds
mean interval time = circulation_seconds / intervals
equivalent sweep time = stored_pairs / updates_per_second
VRAM budget = min(free*fill, free-reserve)
```

BC5 margin workload: 8.26 billion item revisions/s; 127.0 microseconds per interval; 0.683 seconds per equivalent sweep at the average rate. Sweep time is derived. Every trial covered the full field and matched final operator digest and codec-error total across builds.

RG8 margin workload: 6.54 billion item revisions/s; 160.3 microseconds per interval; 0.516 seconds per equivalent sweep at the average rate. Sweep time is derived. Every trial covered the full field and matched final operator digest and codec-error total across builds.

BC5 uses 40% fewer payload bytes than RG8, including B/A, and has additional lossy encoding. For this capacity workload its software mean absolute error is 6.122 token-byte units; RG8 is zero in that byte metric. Neither says how accurate real amplitudes or angles are. Allocator counts establish allocation, not guaranteed physical residency of every page at every moment.

What the hardware counters say

Nsight Compute 2025.1 sampled three evolve-kernel invocations after eight earlier evolve launches: 32 pages of 2048², W=65,536, BC5, sequential kernel launches. The table reports the median per-invocation counter values. This is a 192 MiB field, not the near-capacity experiment.

Counter / meaning	Reference	Optimized
Evolve duration (microseconds)	440.54	104.42
Active-warp occupancy (%)	8.31	51.61
L1/TEX sector hit rate (%)	93.01	59.49
L2 sector hit rate (%)	94.06	84.86
L1/TEX throughput / peak (%)	10.49	45.38
DRAM throughput / peak (%)	1.30	8.21

More hits is not always faster

The optimized kernel has a lower L1/TEX sector hit percentage while finishing substantially sooner. Balanced shared-memory capacity admits more concurrent work, and the pair table trades repeated transcendental math for reads. Cache-hit percentage alone would therefore rank these builds incorrectly.

Three different uses of the word saturation

Quantity	What the evidence supports
Storage occupancy	Roughly 97.6% of the memory free at payload planning was used for payload in the capacity runs.
Cache effectiveness	Sector-hit counters above are measured for the profiled kernels. The configured L2 hitRatio=1 is only a retention preference.
Bandwidth saturation	The sampled optimized DRAM and L1/TEX throughput are below 100% of profiler sustained peak. Full bandwidth saturation is not demonstrated.

Cache flushing and clock control were disabled to avoid claiming cold-cache timings as normal recurrence behavior. Nsight warns that uncontrolled caches and clocks can affect consistency; replay and instrumentation also perturb execution. These counters diagnose the observed bottleneck, while unprofiled repeated timings are the main speed evidence. They do not measure end-to-end PCIe traffic. [R2-R4]

The selected L2 persistence window covers the 1,056,768-byte hot arena, including the pair table. The device reports 36 MiB L2 and a 22.5 MiB maximum persisting allowance. Persistence, shared-memory carveout and cached texture access are policies and mechanisms, not permanent cache locks.

Does it behave like double-slit theory?

It implements identifiable source mechanisms. It does not validate a physical double-slit model. The following diagnostics separate algebraic correctness from the behavior of the encoded recurrence. They run on the native CPU header, plus an explicit double-precision ODE diagnostic. [R1, R5]

An isolated interference comparison

```
a=1, b=exp(i*phi)
I_plus = |(a+b)/sqrt(2)|^2 = 1+cos(phi)
I_minus= |(a-b)/sqrt(2)|^2 = 1-cos(phi)
I_plus + I_minus = 2
```

This is the coherent two-input Hadamard identity before quotient encoding. It is a useful component check, not a spatial slit-and-detector experiment. A detector model would also need normalized probabilities, geometry, propagation, wavelength/path phases and comparison with independent observations.

Executed diagnostic	Observed result
Float Hadamard, all 65,280 nonzero LUT pairs	Maximum relative norm residual: 1.67e-7 before encoding.
Same split followed by LP8	53,440 pairs change energy by more than 1e-4 relative; output/input energy ratios span 3.05e-5 to 32,768.00365.
Opposite-phase cancellation	240/240 directed opposite-phase pairs encode a nonzero sum. Largest raw cancellation amplitude is 1.22e-6; largest decoded sum is 9.51366 (different maxima).
Concrete quotient witness	Input tokens [17,0] produce [254,254] after the encoded split. Tiny nonzero amplitudes can wrap to a large radius.
Source amplitude 2	Encodes as token 176: magnitude 2.37841, phase 11.25 degrees; complex absolute error 0.570964.
Klein seam and inverse words	Maximum sampled seam residual 4.01e-6; zero failures across all 65,536 modular phase inverses.
Fourth-stage forcing	At 256 steps over T=1: classical double RK4 error 1.29e-12; forced map error 2.22e-6 against the unforced reference.

These measurements explain why exact RG8 storage is not an exact wave simulation: LP8 quantization and radius identification already change the physical interpretation. No epsilon cutoff, renormalization or removal of the fourth-stage event was inserted to conceal this. Such changes would define a different binding.

Validation and practical limits

Evidence	Result and scope
Native CUDA self-test	BC5 hardware texture decode, exact RG8 readback, CPU/GPU instruction cases, one-step recurrence, ordered write/read coherence and executable-word mutation pass.
Pair cache on this GPU	All 65,536 token-pair entries at four offsets and two channels match the uncached device calculation.
Core equivalence	8,355,840 BC4 endpoint/sample selector cases; 10,000 BC5 block/error comparisons; all 65,536 LP8 pairs at four offsets; 1,024 complete staged comparisons pass.
Full snapshot equivalence	16 small cases across both codecs: partial windows and CTAs, page/cursor wrapping, whole field, one block, frozen operators, no jitter, inverse gain zero and zero hops.
Graphs and continuation	Both builds checked against sequential full snapshots. Acceptance suite checks repeatability and 7+13 versus 20-interval full snapshot continuation in both codecs.
Memory and synchronization	Compute Sanitizer memcheck, racecheck and synccheck: zero errors/hazards for both codecs on three 48 ² pages, a 257-block partial window and five intervals.
Repeated measurements	Five paired trials for small, larger and common-cache workloads; three paired multi-GiB trials with capacity margin. The failed maximum-capacity repetition is retained.

Boundaries of the result

Full-snapshot comparisons validate the tested digital states; they are not a proof over every possible custom operator image, seed, device or compiler. Floating math can differ on other hardware. The original BC5 build could not create its texture; the reference measurements use the minimal read-mode compatibility fix required to execute it.

The source claim of 24-48 billion stored states is not supported by this format. Even an ideal 12 GiB entirely devoted to payload holds at most 8.590 billion BC5+B/A pairs or 5.154 billion RG8+B/A pairs. Actual usable memory is lower. Updating a fixed-size window is not an O(1) update of the full VRAM field.

Self-mutation demonstrates feedback, not automatic improvement. There is no defined reward function, convergence proof, quantum randomness, guaranteed avoidance of cycles, unrestricted universality proof, or measured zero-PCIe claim. The useful validated artifact is an efficient, editable classical finite-state kernel with a documented source relationship.

Reproduce and extend

Project directory: **Dawnwood_Interactive_RTX5070Ti_v0.3 / Dawnwood_Interactive_RTX5070Ti_v0.3**. The originally supplied path with separate underscore directories did not exist; this matching package is the edited project. Use a fresh output folder for each harness run.

Build and validate

```
.\scripts\build_windows.ps1
.\build\dawnwood.exe --self-test
python tools\gpu_acceptance.py build\dawnwood.exe `
  --out work\acceptance_new
python -m unittest discover -s tests -p "test_*.py" -v
```

A Ninja Release build places executables directly in build; a Visual Studio generator places them in build/Release. The script preserves an existing generator. Tested toolchain: MSVC 19.44, nvcc 12.8.61, sm_120 plus compute_120 PTX, driver 591.59, CUDA driver API 13.1, RTX 5070 Ti Laptop GPU (46 SMs).

Compare and isolate cache policy

```
python tools\benchmark_optimization.py `
  build\dawnwood_reference.exe build\dawnwood.exe `
  --out work\compare_new --pages 32 --page-side 2048 `
  --steps 2048
# Add these to isolate kernel changes at the same preference:
--reference-cache-policy balanced `
--optimized-cache-policy balanced
```

Bounded saturation run

```
.\build\dawnwood.exe --codec bc5 --fill 0.98 `
  --reserve-mib 256 --page-side 4096 `
  --blocks-per-step 65536 --steps 8192 --batch 32 `
  --report work\capacity.json
```

Commands use PowerShell backtick continuation. If local policy requires signed scripts, run the build through powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\scripts\build_windows.ps1; this applies only to that process. Finite --steps is intentional; --steps 0 runs until Ctrl+C. Changing W changes the recurrence.

Editable operator definitions remain in model/operators.json and can be packed with tools/operator_image.py. Snapshot ABI and operator record sizes remain v0.3. The reference binary and cache switches make future comparisons reviewable. The report builder reads saved JSON evidence; it does not substitute theoretical peak specifications for measurements.

Evidence index and references

Files that make the conclusions reviewable

Project-relative evidence	Purpose
results/optimization/benchmark_{small,large,balanced}/benchmark.json	Commands, hardware record, executable hashes, full-state checks and five-trial timing statistics.
results/optimization/benchmark_capacity_margin/benchmark.json	Three paired multi-GiB trials; complete sweep assertion. Original benchmark_capacity records the failed maximum repeat.
results/optimization/capacity_{bc5,rg8}.json	Automatic allocation, exact payload/free bytes and initial capacity-run metrics.
results/optimization/ncu_{reference,optimized}.csv	Three sampled evolve invocations; locale-aware numeric parsing in this PDF builder.
results/optimization/sanitizer_*.txt	Native memory, race and synchronization diagnostics.
results/optimization/theory_diagnostics.json	Independent numerical findings; generated from tools/theory_diagnostics.cpp.
docs/THEORY_SUPPLEMENT.md	Full literal-source map and mathematical supplement.
docs/OPTIMIZATION.md	Changes, reproduction and measurement interpretation.

Primary sources

- [R1] Supplied double-slit-theory.pdf, 24 pages. SHA-256: e3cf7d49e6a4942c7ccad4805f6a2a07e1c817753ff8d384b1f2e37791ebbe27. Original source preserved.
- [R2] NVIDIA CUDA 12.8 Programming Guide: texture/surface coherence and L2 policies. [Official programming guide](#).
- [R3] NVIDIA CUDA 12.8 Runtime API: texture views, cache preference and occupancy queries. [Texture objects](#); [Execution control](#).
- [R4] NVIDIA Nsight Compute Profiling Guide: replay, cache flushing, clocks and metric interpretation. [Official profiling guide](#).
- [R5] Feynman Lectures on Physics, volume III, chapter 1: amplitudes, interference and detection probability. [Quantum behavior](#).
- [R6] Microsoft Direct3D 11 block-compression documentation: BC5 stores two channels in 16 bytes per 4x4 block. [Block compression](#).

Measurements are dated 16 September 2026. Source mathematics, implementation definitions, CPU diagnostics, GPU correctness checks, allocator observations and profiler samples are separate evidence classes throughout this report.